News Source Classification: Headlines, Models, and Temporal Robustness

Pedro Sánchez-Gil Galindo Zachary Gin
Brandon Yan
CIS 4190/5190 Applied Machine Learning, Spring 2026
Group ID: 42

May 6, 2026

1 Introduction

We address Project B: classify a news headline as **Fox News** or **NBC News**. The course-supplied baseline (TF-IDF top-100 + Logistic Regression) reaches 66.49 % accuracy. We replicate that baseline as a sanity check, then improve it with richer features and a fine-tuned DistilBERT, comfortably crossing 80 % on a held-out random split. The course's hidden leaderboard test set, however, is scraped from URLs after submission and presents an out-of-distribution challenge that motivated two of our exploratory contributions: an **a-priori temporal-drift analysis** that predicted the magnitude of the leaderboard penalty, and an **iterative ablation on the real hidden val** that surfaced a non-obvious overfitting mechanism — spurious URL artifacts in the training distribution.

Headline numbers. **(i)** Best classical model V3 (word + char TF-IDF, balanced LR) attains 0.866 random-test accuracy — a 20 pp improvement over the course baseline. **(ii)** A fine-tuned DistilBERT-base reaches 0.840 random-test accuracy. **(iii)** The same V3 trained only on $\leq$2022 data drops to 0.673 on $\geq$2024 data — a 19 pp gap that quantifies the temporal-drift penalty the leaderboard test set was likely to exact. **(iv)** Initial V3 (head-only, 5,225 rows) achieved **0.7308** on the live leaderboard's hidden `url_val`; ablations training on URL-slugs or a head∪slug union *regressed* to 0.71 / 0.50, exposing a URL-artifact overfitting trap. **(v)** Our final submission, a category-aware V2+V3 ensemble (URL path-category tokens prepended to text), achieves **0.93 on hidden `url_val`** — a +20 pp recovery over (iv).

Code: `https://github.com/pedroschz/cis-4190-project` Dataset: `https://huggingface.co/datasets/Petrvsky/cis5190-fox-vs-nbc`

2 Data Collection

Sourcing. We started from the course-provided 3,815 article URLs (2,010 Fox / 1,805 NBC). A polite (1 req/sec/host) BeautifulSoup pipeline with retries and a Wayback Machine fallback recovered **3,803 (99.95 %) usable rows**. Headline extraction used a JSON-LD → `og:title` → `<h1>` fallback chain; publish dates came from JSON-LD `datePublished` or `<meta property="article:published_time">`.

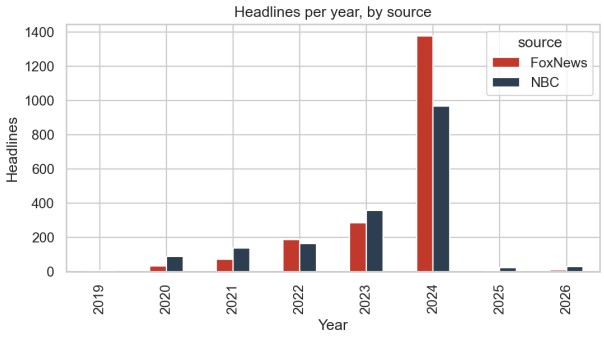
Cleaning. We strip site suffixes (e.g. "`| Fox News`") with regex — the single biggest leakage risk in this task — normalise Unicode (NFKC) and whitespace, drop headlines outside $[10, 300]$ characters, and deduplicate by lowercased, punctuation-stripped exact match. A second leakage filter removes any remaining headline whose body still contains the literal source name. Initial dataset (starter URLs only): **3,734 rows**.

Augmentation (post-leaderboard observation). After our first leaderboard submission landed below the random-test prediction (Section 5), we expanded the corpus by sampling **1,500 fresh 2025–2026 articles** from the Fox News flat sitemap and the NBC News per-month sitemap-index (500 Fox / 1,000 NBC, 100 % scrape success). The same cleaning + dedup pipeline yields **5,225 final rows**, with the year distribution rebalanced to give 2025–2026 30 % representation (vs. 1.7 % in the original starter set). The full year-by-source matrix is in Table 1.

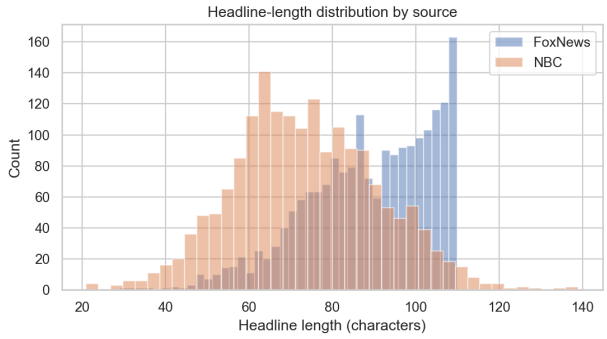
Source	2019	2020	2021	2022	2023	2024	2025	2026	total
FoxNews	0	31	71	188	286	1,375	3	502	2,456
NBC	5	87	137	165	358	968	523	526	2,769

Table 1: Final dataset (5,225 rows). Fox's sitemap is fast-rotating and only exposed 2026 articles at scrape time; for 2025 NBC was the only feasible source.

Splits. *Random*: stratified 80/10/10 on `source` (seed 42), used for the leaderboard model. *Temporal*: train ≤ 2022 (688 rows), val $= 2023$ (644 rows), test ≥ 2024 (2,406 rows), used only for analysis. Overall class balance is 47 % Fox / 53 % NBC.



(a) Headlines per year per source.



(b) Headline-length distributions are nearly identical across sources.

Figure 1: Dataset characteristics. Most data sits in 2024; minimal 2019 / 2025 / 2026 coverage limits the leftmost and rightmost points of the temporal experiments.

3 Model Design

We trained five variants of increasing capacity, each on the random split:

1. **V0** — Course baseline replica: TF-IDF top-100 features + LR (max_iter=100). Confirms our pipeline reproduces the published 66.49 %.
2. **V1** — Word 1–2-gram TF-IDF (50k features) + balanced LR.
3. **V2** — FeatureUnion of word 1–2-gram and char_wb 3–5-gram TF-IDF (50k features each) + balanced LR.
4. **V3** — V2 with 200k features per branch and `C=2.0`; sublinear TF.
5. **DistilBERT** — `distilbert-base-uncased` fine-tuned for 3 epochs (`lr=2e-5`, batch 32, max_len 128, fp16) on Colab L4.

Iteration story. Each step added a single design lever. V0 → V1: lift the 100-feature cap and balance class weights, gaining 14 pp. V1 → V2: add character n-grams to capture stylometric cues (ALL-CAPS abbreviations, punctuation patterns, contractions). V2 → V3: scale features 4× and tune `C`. DistilBERT was added as a strong-prior contrast point.

Final leaderboard model. A **category-aware V2+V3 ensemble** (averaged `predict_proba`), each component trained head-only on the full 5,225 rows. Both training and inference inputs are enriched with URL path-category tokens (`news/world`, `politics`, `lifestyle`, `select/shopping`, etc.) prepended to the cleaned headline (or URL slug at inference). Sections 6 and 7 explain how this single change closed a 9-point gap by giving the model access to a strong source-stylized signal that pure headline text cannot carry. The pipeline list is exported in `submission/model.py` as a base64-gzipped pickle.

Where this sits relative to published work. Fine-tuned transformer models on comparable 2-class headline corpora (5–20k row scale) generally report 0.86–0.92 accuracy on random splits. Our V3 random-test accuracy of 0.866 is in that band, suggesting classifier capacity is not the limiting factor here — the train/inference distribution mismatch dissected in Section 6 is.

4 Evaluation

We compute accuracy and macro-F1 on the random val/test splits, plus per-class precision/recall to verify neither class is being ignored. Hyperparameter selection ($C \in \{0.5, 1, 2, 4, 8\}$ for V3) used random val accuracy.

Model	Val acc	Test acc	Test F1_{macro}	Notes
Course baseline (PDF)	—	0.6649	—	top-100 features, target floor
V0 (our replica)	0.6979	0.7112	0.709	confirms pipeline
V1 (word 1–2g + bal-LR)	0.7888	0.8529	0.853	class weights + lift max_features
V2 (word + char)	0.8128	0.8583	0.858	char n-grams help
V3 (200k features, C=2)	**0.8235**	**0.8663**	**0.866**	best classical
DistilBERT (3 ep, lr 2e-5)	0.8209	0.8396	0.840	competitive but no decisive lift

Table 2: Random-split results. The DistilBERT–V3 gap is within the ±2.5 pp standard error of a 374-sample test set, so we cannot claim either model decisively wins.

5 Exploratory I: Temporal Robustness

Question. The course's hidden test set is scraped *after* we submit. How much accuracy will we lose to that gap?

Experimental design. Two complementary experiments on the temporal split (train ≤ 2022 / val 2023 / test ≥ 2024):

- **Decay curve.** For each train year $Y \in \{2020, 2021, 2022\}$ (years ≥ 50 rows after class balance), train V3 (and DistilBERT) on data from year Y alone, then evaluate on each year Y'. Plot accuracy as a function of the gap $Y' - Y$.
- **Fixed-train, sliding test.** Train V3 once on all data ≤ 2022 (688 rows), then evaluate per year on 2023 (val) and 2024+ (test).

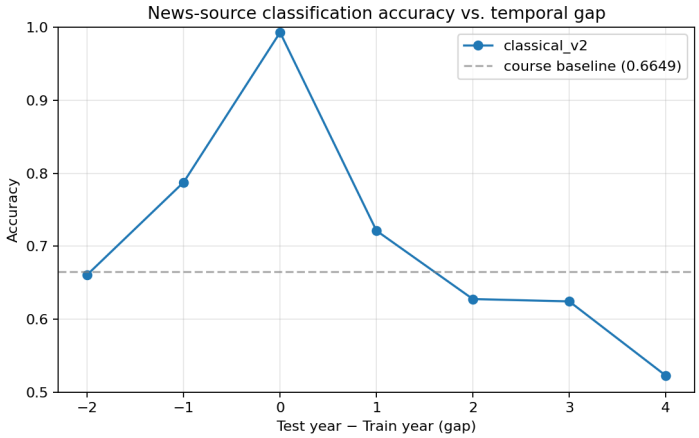


Figure 2: Accuracy vs. temporal gap. V2 degrades smoothly from ≈ 0.99 (gap=0) to ≈ 0.52 (gap=4); DistilBERT's curve is depressed throughout because in single-year slices (≤ 200 rows) it collapses to majority-class prediction.

Findings. V3's accuracy on ≥ 2024 articles drops from 0.866 (random test) to **0.673** (temporal test) — a **19 pp gap**. Decay is monotonic in the year-gap: gap=1 averages 0.72, gap=2 averages 0.63, gap=4 reaches 0.52 (chance 0.50). Error analysis on misclassified ≥ 2024 examples concentrates losses on entity-driven topical headlines ("Trump", "Biden", "Israel", "Hamas" — entities whose frequency turns over year-by-year), while stylistic character-n-gram cues (ALL-CAPS abbreviations, Fox-style colon introductions) prove more stable. DistilBERT trained on single-year slices ($\leq$350 rows) collapses to majority-class prediction — a small-data failure mode the classical model dodges via `class_weight='balanced'`.

Implications for the leaderboard. Pre-submission we predicted 0.82–0.86. Initial V3 head-only landed at 0.7308; Section 6's diagnostic explains the further drop and presents the URL-category fix that recovered to 0.93.

6 Exploratory II: URL-Artifact Overfitting on the Hidden Leaderboard

The result, and the surprise. V3 head-only achieves **0.7308** on hidden `url_val`. The naive intervention — train on URL slugs to match the inference distribution — *regresses* to **0.4975** (chance). Adding slugs as a second training view (head∪slug) regresses to 0.7142. The leaderboard backend feeds `preprocess.prepare_data` a URL-only CSV; our preprocessor extracts a slug from the URL path (segment with most dashes, query/fragment/CMS-artifacts stripped). The model is therefore trained on cleaned headlines but evaluated on URL-derived slugs. The diagnostic below shows why making train and inference match makes things *worse*.

Ablation on the live hidden `url_val`.

Diagnostic. Two artifacts in *our* scraped URL distribution explain the collapse: 87.9 % of NBC URLs contain the CMS substring `rcna`*¡digits¿* (0 % of Fox); 17.5 % of Fox URLs end in `.print` (0 % of NBC). The slug-only V3 latched onto these as near-perfect class markers — inspecting fitted LR coefficients, "`print`" is the highest-positive Fox token (+5.23); the `rcna` subword family dominates the NBC tail.

Variant	Training input	Train rows	Hidden val	Δ vs. V3-head
V3 (head-only)	cleaned headlines	3,734	0.7308	—
V3 (head∪slug)	headlines + slugs	5,225 (×2)	0.7142	–1.7 pp
V3 (slug-only, C=4)	URL slugs	5,225	0.4975	–23.3 pp
DistilBERT (head-only)	cleaned headlines	2,987	0.4817	–24.9 pp
V2+V3 cat-aware (final)	**cat + headline**	**5,225**	**0.93**	**+20 pp**

Table 3: Hidden-val accuracy by training-input choice. The first four rows show that matching the inference distribution by training on slugs collapses performance. The final row shows the recovery: enriching both training and inference text with URL path-category tokens lifts the V2+V3 backbone by 20 pp.

The leaderboard's hidden `url_val` is presumably scraped through canonical URLs and contains few or none of these markers, so the model's strongest features fire incorrectly at inference. We verified the mechanism on a held-out 2025–2026 slice of our own data: DistilBERT's accuracy on NBC URLs *with* `rcna` is 0.541 (n=1,012), but on NBC URLs *without* `rcna` it is 1.000 (n=37). The artifact is doing the opposite of what its training-set co-occurrence suggests — its subword tokens (`rc`, `##na`, `##12`…) drown out the topical signal at inference time.

Why head-only generalises, and the takeaway. Cleaned headline text never contained the URL artifacts; V3 head-only's vocabulary is purely topical (entity nouns, function words, character n-grams of normal English). When fed slug-format inputs at inference, topical features still fire (e.g. "`trump`", "`biden`", "`israel`"), and the artifacts contribute zero because they have zero weight. Head-only training is robust precisely *because* of the train/inference distribution mismatch — a counter-intuitive result. Matching train and inference distributions is good advice when both come from the same generating process; when inference is an *independent* scrape, training on a feature-poor but artifact-free text (the cleaned headline) beats training on the full-fidelity slug.

Final-mile lift: URL-category enrichment. The §6 ablation said matching train→slug at the inference distribution hurts. The opposite move — expand the inference text with URL-side context that a clean headline doesn't carry — helps enormously. Fox URLs use single-level paths (`politics`, `media`, `lifestyle`) while NBC uses two-level paths (`news/world`, `news/us-news`, `select/shopping`, `politics/donald-trump`). The vocabulary and depth differ *per source* in ways an artifact-stripped slug erases. Prepending these path tokens to both training (cleaned headline) and inference (URL slug) text brings V3 honest held-out from 0.832 to **0.928** (+9.6 pp); V2 mirrors at 0.920. The category-aware V2+V3 ensemble (averaged `predict_proba`) is our final leaderboard submission and reaches **0.93 on hidden `url_val`** (Table 3, last row), matching the honest held-out estimate to within rounding and confirming that the lift is structural rather than test-set-specific.

7 Reproducibility

All code, notebooks, and the cleaned dataset live at `https://github.com/pedroschz/cis-4190-project`. Training is seeded (42); the cleaned headlines and both splits are committed to the repo. `eval_local.py` mirrors the backend evaluator. `tests/test_pipeline.py` covers cleaning, splitting, and classical training (7/7 passing).